

REPO SUMMARY

CODEGO Internal Records App

Evidence-based one-page summary derived from the Flask app, Docker setup, SQL init files, templates, and services in this repository.

WHAT IT IS

A Flask web application for managing industrial district settlement records and related legal process data. The repo shows role-based internal workflows, MySQL-backed CRUD screens, PDF report generation, audit logging, and password reset by email.

WHO IT'S FOR

Primary users appear to be CODEGO internal staff in settlement (`assent`), legal (`jur`), manager, and admin roles who maintain company/lot records and legal case information.

WHAT IT DOES

- Authenticates users and routes them to settlement or legal dashboards by department/role.
- Registers company and lot records in `municipal_lots` with server-side validation.
- Uploads company images and descriptions into `empresa_infos`.
- Registers and edits multiple legal processes per company in `processos`.
- Generates branded PDF reports with company details and linked legal processes.
- Tracks edits and registrations in an admin-viewable `logs` table.
- Supports user registration plus email-based password recovery/reset.

HOW IT WORKS

UI layer: Flask blueprints render Jinja templates such as `login.html`, `menu.html`, `cadastro.html`, `editar.html`, and report screens.

App layer: `create_app()` loads env vars, config, bcrypt, and registers auth, dashboard, cadastro, relatorios, logs, and edicao blueprints.

Service layer: Auth, token, email, cadastro normalization, logging, and PDF watermark helpers handle password hashing, reset tokens, SMTP sends, validation, and report formatting.

Data flow: Browser form -> blueprint -> service validation -> MySQL via `mysql.connector` -> optional image upload to `uploads / app/static/imagens_empresas` -> HTML or PDF response.

Core tables: SQL init files define `usuarios`, `municipal_lots`, `empresa_infos`, `logs`, and `processos`.

Runtime topology: `docker-compose.yml` runs `web` (Gunicorn on 8000), `db` (MySQL 8), and `nginx` (port 80 reverse proxy).

HOW TO RUN

- 1 Use the existing `.env` or provide at least `SECRET_KEY`, `DB_HOST`, `DB_PORT`, `DB_PASSWORD`, `DB_NAME`, `SMTP_USER`, and `SMTP_PASS`.
- 2 Start the stack with `docker compose up --build`. The compose file initializes MySQL from `docker/mysql/init` and starts Nginx in front of the Flask app.
- 3 Open `http://localhost`. Nginx proxies requests to Gunicorn on port `8000`; uploads are mounted from `./uploads`.
- 4 Local non-Docker dev instructions, automated tests, and a README-based setup guide: **Not found in repo.**

Evidence sources: `app/__init__.py`, route/service modules, `docker-compose.yml`, `Dockerfile`, `nginx/nginx.conf`, `docker/mysql/init/*.sql`.

Generated for repo snapshot: 2026-04-13